



UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN
GRÁFICA E INTERACCIÓN HUMANO
COMPUTADORA



TAREA: TOROIDE

NOMBRE: ANDREA MATA RAMÍREZ

Nº CUENTA: 319052277

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 04 / 05 / 2026

CALIFICACIÓN: _____

ACTIVIDAD:

Crear un toroide, con geometría (no debe tener huecos), con texturizado (debe tener textura continua que envuelve "una sola vez") y con iluminación (debe interactuar con las luces normales)

DESARROLLO:

En el código primero se declaran variables globales para ventana, mesh, cámara, materiales (brillante y opaco), variables para controlar el tiempo y limitar a 60 FPS.

ToroideConfig (Figura 2) agrupa toda la información del toroide, se declaran resolución horizontal (resHor), resolución vertical (resVert), radios interior y exterior, posición en el mundo, escala, puntero al material, en este caso todos los toroides fueron brillantes para mostrar la interacción con luces, y finalmente se declara el índice del meshList donde está la malla.

CreateObjects Crea el piso de la escena

CreateSHders Lee los archivos .frag y .vert del shader

generarMeshToroide (Figura 3 y 3.1) esta parte es muy importante, aquí se genera la geometría del toroide usando la ecuación paramétrica (Figura 1):

```
/*
 * se usa ec. paramétrica:
 *
 *      X(phi, theta) = (R + r*cos(phi)) * cos(theta)
 *      Y(phi, theta) = (R + r*cos(phi)) * sin(theta)
 *      Z(phi, theta) = r * sin(phi)
 *
 *      R = radioExt
 *      r = radioInt
 *
 * la normal en cada vertex apunta desde dentro hacia afuera
 * n = (cos(phi)*cos(theta), cos(phi)*sin(theta), sin(phi))
 *
 * vertices almacenan: pos(3) + uv(2) + normal(3) = 8 floats
 */
```

Figura 1. Ecuación paramétrica

Por cada vértice se almacenan 8 valores: Posición (x, y, z), textura (u, v) y normal (nx, ny, nz) n es la normal, en la figura anterior se incluye la forma de calcularla, la normal ya es unitaria por construcción, por lo que no es necesario que se normalice.

Por cada par de anillos se forman rectángulos formados por dos triángulos, al final se crea un mesh con los datos y lo retorna.

agregarToroide (Figura 4) Esta función recibe todos los parámetros del toroide, llama a generarMeshToroide para crear la malla, la agrega al meshList y guarda todo en toroides. MeshIndex guarda en qué posición del meshList está la malla para la hora del render. +

En el main se inicializa la ventana y posteriormente se hace el registro de los toroides, en este caso se hicieron 4 toroides con valores diferentes de resolución, proporción y posición para mostrar cómo afecta cada rubro (Figura 5).

En el loop principal (el while):

Tiempo: calcula deltaTime para que el movimiento pueda ser independiente de los FPS

Eventos: procesa teclado y mouse

Cámara Actualiza posición y orientación según teclas y mouse

Skybox

Uniforms: se envía a GPU matrices de proyección, vista y posición del observador

Linterna: actualiza la posición del Spotlight con la posición de la cámara

Piso: se dibuja con el tipo de material (opaco) y textura

Toroides: itera sobre toroides, construye la matriz model con translate + rotate + scale y se llama a RenderMesh (Figura 6)

```
struct ToroideConfig {
    int      resHor;
    int      resVert;
    GLfloat  radioInt;
    GLfloat  radioExt;

    glm::vec3 posicion;
    GLfloat  rotacionX;
    glm::vec3 escala;
    Material* material;
    int meshIndex;
};
```

Figura 2. Configuración

```
// indx
for (int i = 0; i < resHor; i++) {
    unsigned int k1 = i * (resVert + 1);
    unsigned int k2 = (i + 1) * (resVert + 1);

    for (int j = 0; j < resVert; j++) {
        indices.push_back(k1);
        indices.push_back(k2);
        indices.push_back(k1 + 1);

        indices.push_back(k1 + 1);
        indices.push_back(k2);
        indices.push_back(k2 + 1);

        k1++; k2++;
    }
}
```

Figura 3.1 (continuación de
función)

```
Mesh* generarMeshToroide(int resHor, int resVert, GLfloat radioTubo, GLfloat radioExt)
{
    std::vector<GLfloat> vertices;
    std::vector<unsigned int> indices;

    const GLfloat paso_phi = (2.0f * glm::pi<GLfloat>()) / resHor;
    const GLfloat paso_theta = (2.0f * glm::pi<GLfloat>()) / resVert;

    // vertices
    for (int i = 0; i <= resHor; i++) {
        GLfloat phi = i * paso_phi;
        GLfloat cosPhi = glm::cos(phi);
        GLfloat sinPhi = glm::sin(phi);

        for (int j = 0; j <= resVert; j++) {
            GLfloat theta = j * paso_theta;
            GLfloat cosTheta = glm::cos(theta);
            GLfloat sinTheta = glm::sin(theta);

            // ec parametrica
            GLfloat x = (radioExt + radioTubo * cosPhi) * cosTheta;
            GLfloat y = (radioExt + radioTubo * cosPhi) * sinTheta;
            GLfloat z = radioTubo * sinPhi;

            // posicion
            vertices.push_back(x);
            vertices.push_back(y);
            vertices.push_back(z);

            // coordenadas para textura
            vertices.push_back((GLfloat)j / resVert);
            vertices.push_back((GLfloat)i / resHor);

            // normal
            vertices.push_back(-cosPhi * cosTheta);
            vertices.push_back(-cosPhi * sinTheta);
            vertices.push_back(-sinPhi);
        }
    }
}
```

Figura 3. Geometría

```

void agregarToroide(
    int resHor, int resVert, GLfloat radioInt, GLfloat radioExt, glm::vec3 posicion,
    GLfloat rotacionX, glm::vec3 escala, Material* material
){
    ToroideConfig cfg;
    cfg.resHor = resHor;
    cfg.resVert = resVert;
    cfg.radioInt = radioInt;
    cfg.radioExt = radioExt;
    cfg.posicion = posicion;
    cfg.rotacionX = rotacionX;
    cfg.escala = escala;
    cfg.material = material;
    cfg.meshIndex = (int)meshList.size();
    meshList.push_back(generarMeshToroide(resHor, resVert, radioInt, radioExt));
    toroides.push_back(cfg);
}

```

Figura 4. Parámetros

```

// resHor, resVert, radioTubo, radioExt
agregarToroide(
    50, 50, 0.5f, 2.0f,
    glm::vec3(0.0f, 10.0f, 10.0f), -90.0f, glm::vec3(3.0f),
    &Material_brillante);

agregarToroide(
    60, 20, 1.0f, 3.0f,
    glm::vec3(0.0f, 20.0f, 0.0f), -90.0f, glm::vec3(3.0f),
    &Material_brillante);

agregarToroide(
    30, 60, 2.0f, 4.0f,
    glm::vec3(0.0f, 30.0f, -15.0f), -90.0f, glm::vec3(3.0f),
    &Material_brillante);

agregarToroide(
    20, 20, 3.0f, 5.0f,
    glm::vec3(0.0f, 40.0f, -40.0f), -90.0f, glm::vec3(3.0f),
    &Material_brillante);

```

Figura 5. Registro

```

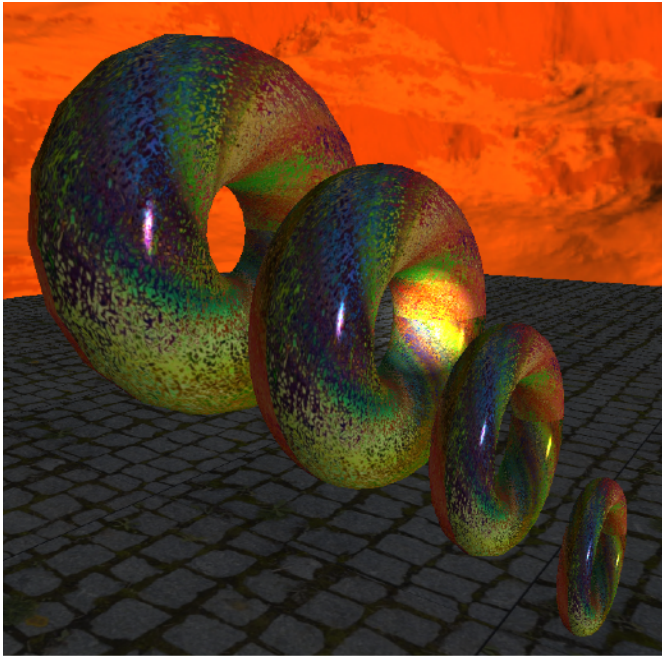
// toroides
ToroideTexture.UseTexture();
glUniform3fv(uniformColor, 1, glm::value_ptr(glm::vec3(1.0f)));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(glm::vec2(0.0f)));

for (const ToroideConfig& t : toroides) {
    model = glm::mat4(1.0f);
    model = glm::translate(model, t.posicion);
    model = glm::scale(model, t.escala);
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    t.material->UseMaterial(uniformSpecularIntensity, uniformShininess);
    meshList[t.meshIndex]->RenderMesh();
}

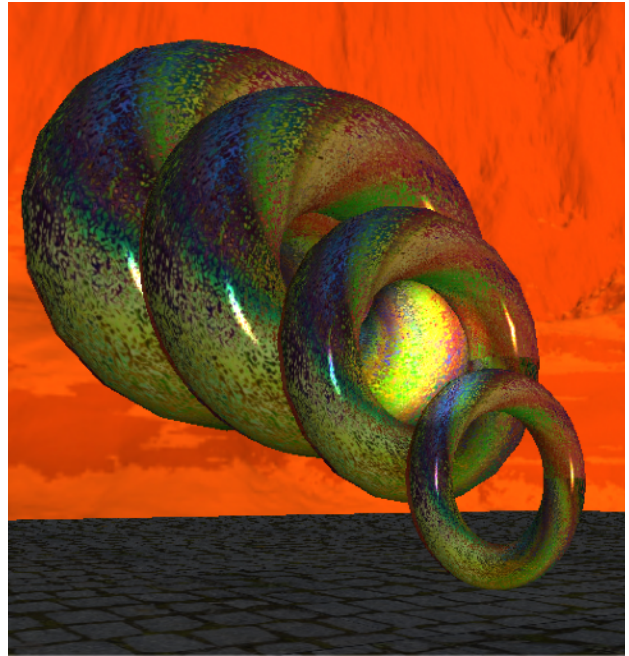
```

Figura 6. Render

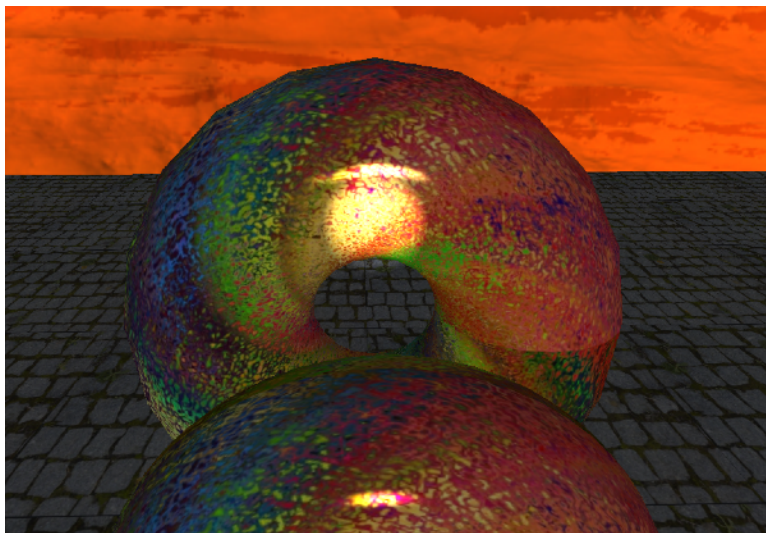
El resultado fue el siguiente



Luces interactúan bien con toroide



Textura bien aplicada, no huecos



Diferencia por parámetros como resolución y radio

COMENTARIO:

Se logró la actividad, me pareció interesante visualmente cómo afecta la resolución del toroide y tuve algunos problemas con la función pero lo pude resolver.